

Analytics Dashboard

Civic intelligence — from raw reports to predictive area, department, and city-level intelligence.

This document is the consolidated feature reference for the StreetLight-PK Analytics Dashboard. It covers the three role-based dashboard views (Department Officer, City Admin, Super Admin), the data architecture, every KPI and chart panel, the AI insight cards, the predictive alerts engine, and a free-tier implementation roadmap. Each module is documented with its purpose, computation logic, and zero-cost build path.

Contents

- | | |
|-----|--|
| 1. | UI Mockup |
| 2. | Architecture & Data Flow |
| 3. | Dashboard Views (Officer · City Admin · Super Admin) |
| 4. | KPI Stat Cards |
| 5. | Chart Panels |
| 6. | AI Insight Cards |
| 7. | Predictive Alerts & Recommendations Feed |
| 8. | Citizen Health & Fraud Geography |
| 9. | Action Buttons |
| 10. | Implementation Roadmap (All Free) |

1. UI Mockup

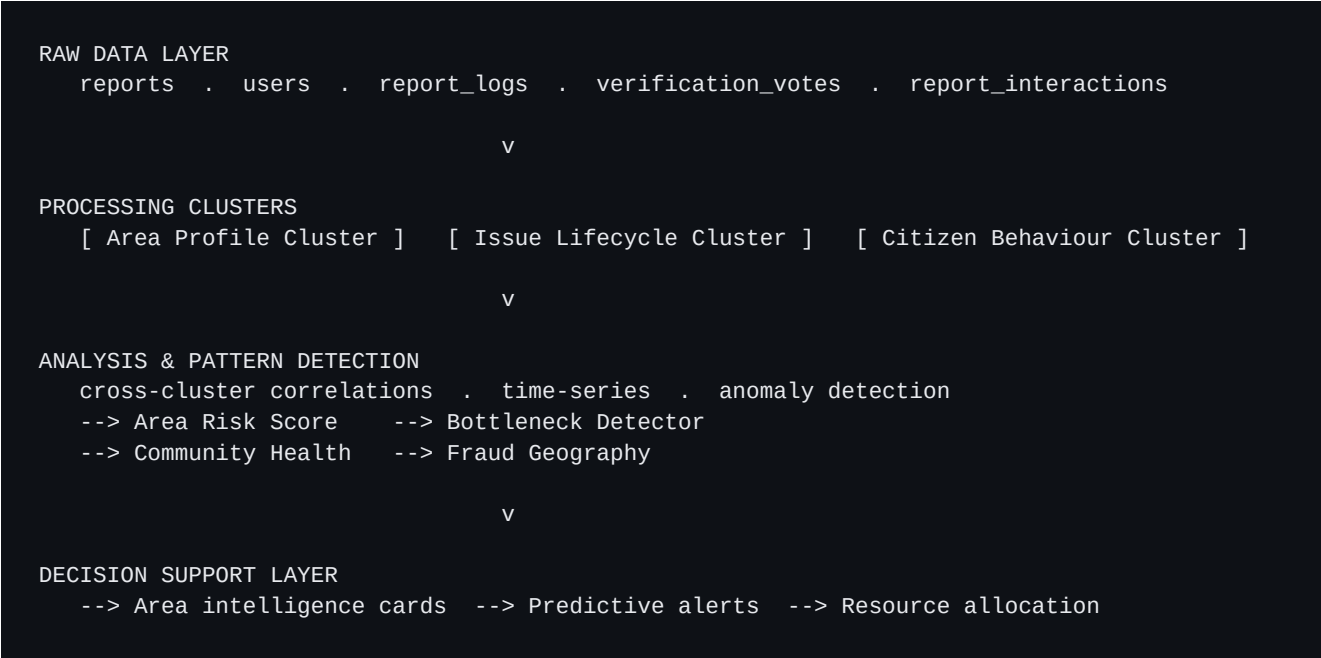
The dashboard is a single-screen surface designed for fast scanning. KPI stat cards sit at the top, chart panels in the middle, AI insight cards beneath, and the predictive alerts row anchors the bottom. The mockup below shows the view for a high-risk area selection.



Full analytics dashboard for a selected area (Gulberg III — high risk). This is the canonical UI shown to all three roles; only the underlying data scope changes by role (see Section 3).

2. Architecture & Data Flow

The dashboard transforms raw complaint data into actionable intelligence through three processing clusters that feed a unified decision-support layer. Every metric on the screen can be traced back through this pipeline to the source rows in Supabase.



Cluster responsibilities

Cluster	Reads from	Produces
Area Profile	reports, geofences	Density, recurrence, hot-zone polygons, area-level totals
Issue Lifecycle	reports, report_logs	Avg time per status stage, bottleneck detection, SLA breaches
Citizen Behaviour	users, verification_votes, report_interactions	Returning-reporter %, badge tiers, vote participation, fraud signals

3. Dashboard Views

The dashboard renders three distinct views depending on the signed-in role. Each view is a strict superset of the one below it: officers see their patch, city admins see their whole city, and super admins see the country. The same KPI and insight components are reused — only scope and aggregation change.

DEPARTMENT OFFICER VIEW

Sees only their own city and department. One zone tab is shown and is non-clickable. The view includes the four KPI cards, the issue-breakdown donut, the pipeline lifecycle funnel, the three AI insight cards, the predictive alerts feed scoped to their patch, and a Generate PDF Report button.

CITY ADMIN VIEW

Sees their entire city. Can switch between department tabs (e.g. Highways and LWMC). Same KPI cards and charts as the officer view, but aggregated across all departments in the city. The bottleneck signal here surfaces **which department** is dragging the city average down, not just which stage.

SUPER ADMIN VIEW

Sees all of Pakistan. City-wide aggregate KPIs sit at the top, followed by side-by-side city summary cards (one for Lahore, one for Karachi, etc.). Each summary card carries the City Admin name, a risk badge, mini KPIs, and a 30-day forecast. Cross-city alerts are ranked by severity at the bottom of the screen.

What changes by role (same UI, different data scope)

The layout above is identical for all three roles — the same KPI cards, the same donut and funnel, the same insight cards, the same alerts row. What changes is the **scope of the data** filling those components. The frontend doesn't change at all; only the ?scope= parameter on each API call differs.

UI element	Department Officer	City Admin	Super Admin
Top tabs	One tab (their assigned zone), non-clickable	Department tabs (Highways, LWMC)	City tabs (Lahore, Karachi, ...)
Risk badge	Zone risk (e.g. Gulberg III — HIGH RISK)	City + dept risk (e.g. Lahore LWMC — MODERATE)	National risk (Pakistan — STABLE)
Total Reports (30d)	Zone count	City total across all departments	National total
Avg Resolution Time	Zone average	City average, weighted across departments	National average, weighted across cities
Re-report Rate	Zone re-report %	City re-report %, broken down by dept	National re-report %, broken down by city
Issue Type Donut	Zone breakdown	City-wide breakdown	National breakdown
Pipeline Funnel	Zone stage timings	City stage timings; surfaces which dept is the bottleneck	National stage timings; surfaces which city is the bottleneck

UI element	Department Officer	City Admin	Super Admin
Bottleneck Signal	Field team delay in their zone	Names the slowest department	Names the slowest city
Predictive Alerts	Only alerts for their zone	All alerts in their city, ranked by severity	Cross-city alerts, ranked by severity
Generate Full Report	Zone-level PDF	City-level PDF with department breakdowns	National PDF with city breakdowns

Implementation note

Tab clicks set a React state value `selectedScope`. Every analytics API call appends the appropriate filter (`?zone=gulberg_iii`, `?city=lahore&dept=lwmc`, or `?scope=national`). The backend applies a row-level filter on Supabase and returns the same response shape regardless of scope, so a single set of components renders all three views. Zero additional cost.

4. KPI Stat Cards

Four KPI cards anchor the top of every dashboard view. Each card carries a headline number, a delta against the prior comparable window, and a colour-coded signal so an admin can read the city's health in a single glance.

Metric	What it shows	Delta logic	Red-alert threshold
Total Reports (30d)	Count of complaints submitted in the last 30 days	% change vs prior 30-day window	Spike >20% above last month
Avg Resolution Time	Mean days from PENDING to RESOLVED	+/- vs last month average	Above 5 days (city target)
Re-report Rate	% of resolved complaints that get re-reported within 30 days	Percentage-point change vs prior month	Above 25% — fixes not holding
Community Score	0–100 composite of citizen engagement and vote quality	+/- vs last month	Below 40 — verification failing

Implementation

All four metrics are SQL aggregates over the existing Supabase reports and users tables. The endpoint `GET /admin/analytics/kpi?scope=X&days=30` returns the headline value, the delta, and a precomputed signal colour so the frontend stays presentation-only.

5. Chart Panels

Four chart panels sit below the KPI row. Each one is built with free, MIT-licensed libraries — no paid BI service required.

Issue Type Breakdown

Donut chart — Pothole vs Garbage vs Streetlight vs Other. Click a segment to filter the map to that type.

Free: Recharts PieChart (MIT)

Pipeline Lifecycle (Avg Days)

Horizontal funnel showing average days at each stage: Pending → Under Review → Dispatched → In Progress → Resolved. Exposes where time is lost.

Free: Recharts BarChart (horizontal)

Weekly Trend Line

7-day rolling complaint count. Spots day-of-week patterns and event-driven surges (rain, public holidays).

Free: Recharts LineChart

Heatmap Calendar

GitHub-style daily complaint heatmap over the last 90 days. Reveals seasonal surges (monsoon potholes, winter streetlight faults).

Free: react-calendar-heatmap (MIT)

6. AI Insight Cards

Three insight cards convert raw aggregates into plain-language recommendations. All three run on free Python rules — no paid ML API is required.

■ Next-30d Forecast

What it shows. Predicts how many complaints to expect over the next 30 days for the selected scope.

Computation. 7-week moving average of weekly complaint counts, multiplied by a seasonal index derived from the same calendar week in prior years. Surfaces a +/- delta against the current 30-day baseline.

Example: Lahore pothole complaints spike every monsoon — this card warns two weeks early so field teams can be pre-positioned.

■ Bottleneck Signal

What it shows. Identifies which pipeline stage is currently dragging the average resolution time.

Computation. SQL query that computes average dwell time per status stage over the last 30 days and returns the stage with the maximum value, along with a diagnostic message.

*Example: If IN_PROGRESS shows a 5.1-day average stall, the card reads: **Field team delay suspected.***

♥ Citizen Health Index

What it shows. A 0–100 score that captures whether citizens are still engaging with the platform in a healthy, trust-building way.

Computation. Weighted composite of: returning-reporter %, average impact score, verification participation rate, and Paragon-tier citizen count.

Example: A score below 50 means citizens are losing trust in the resolution process — re-engagement campaign recommended.

7. Predictive Alerts & Recommendations Feed

Five alert types fire automatically based on Python rules running over the same aggregates that power the KPI cards. No paid ML service is involved. Results are cached in an `analytics_alerts` table and re-computed on dashboard load or on a daily cron.

■ Area Warning (red)

Fires when: Re-report rate above 30%

"Re-report rate 31% — field verification recommended"

■ Resource Suggestion (blue)

Fires when: Pending count above 50 AND a rising forecast

"Assign 2 extra field workers this week"

■ Seasonal Prediction (amber)

Fires when: Within 3 weeks of a historical peak period

"Pothole spike expected — monsoon approaching"

■ Department Nudge (orange)

Fires when: Any category avg resolution above 10 days

"Garbage avg 12.4d — target is 5d"

■ Anomaly Alert (red bold)

Fires when: Single-day volume spikes 3× above the 30-day average

"Lahore volume spiked 3× today"

Implementation

Each alert is a pure Python function that takes the latest aggregate row and returns either `None` or an alert object `{type, severity, message, scope, fired_at}`. The functions are composed into a single rule engine that runs on dashboard load (cached for 10 minutes) and on a nightly cron. Alerts persist in Supabase so historical alert volume itself becomes a metric the Citizen Health Index can read.

8. Citizen Health & Fraud Geography

Beyond the headline Citizen Health Index, four dedicated panels expose where engagement and trust are drifting. These run on the same users, verification_votes, and report_interactions tables — no new sources are required.

■ **Badge Tier Distribution**

Bar chart showing Bronze / Silver / Gold / Paragon citizen counts per scope. More Paragon-tier reporters means more reliable reports.

■ **Fraud Geography**

Flags areas with concentrated GPS spoofing and duplicate submissions. Helps identify coordinated abuse patterns and single-account spammers.

■ **Engagement Drop-off Tracker**

Tracks areas where returning-reporter counts are declining — an early warning that citizens are losing trust in the resolution process before it shows up in re-report rate.

■ **Verification Participation**

% of reports that received a community vote. Areas with low participation need a re-engagement campaign before the Citizen Health Index drops.

9. Action Buttons

Three actions live in the bottom row of every view. They turn the dashboard from a passive monitor into a working tool.

Button	What it does	Free implementation
Generate Full Report	Exports all KPIs, charts, and AI insights as a PDF for sharing in weekly municipal meetings.	Backend renders an HTML template, then converts it via a headless PDF engine (the same path used to produce this document).
Get Recommendations	Returns a structured action plan: which departments to nudge, where to deploy field teams, and which seasonal prep is due.	Python rule engine reads the latest analytics_alerts rows and returns a formatted JSON payload for the frontend.
Download Raw Data	CSV export of the reports table for the current scope and filter state, for offline analysis.	GET /admin/analytics/export?scope=X&format=csv served as a FastAPI StreamingResponse.

10. Implementation Roadmap (All Free)

The build sequence below moves from the highest-value, lowest-risk pieces to the more ambitious ones. Every item uses Supabase (free tier), FastAPI, Recharts (MIT), and standard Python libraries. There is no paid ML, BI, or GIS dependency anywhere in the plan.

#	Priority	Feature	Frontend	Backend
1	HIGH	KPI stat cards (real data)	Replace static values with API call	GET /admin/analytics/kpi — SQL aggregates
2	HIGH	Issue type donut chart	Recharts PieChart	GROUP BY issue_type query
3	HIGH	Pipeline lifecycle funnel	Recharts horizontal BarChart	Avg days per status stage SQL
4	MED	Bottleneck signal card	Card component, data from API	Max stall stage detection query
5	MED	30-day forecast card	Card component, data from API	7-week moving average + seasonal index (Python)
6	MED	Citizen Health Index	Score card component	Weighted formula across users + report_logs
7	LOW	Predictive alerts feed	Scrollable alert list	Rule engine + analytics_alerts table
8	LOW	Heatmap calendar	react-calendar-heatmap	Daily count query (last 90 days)
9	LOW	Area risk score badge	Badge chip with colour	Nightly batch scoring script
10	NICE	PDF & CSV export	Download button	Headless PDF renderer / StreamingResponse

Total external cost: \$0. Every feature in this roadmap runs on Supabase (free tier), FastAPI, Recharts, react-calendar-heatmap, and standard Python. There is no paid ML, BI, or GIS service anywhere in the build path — the dashboard ships entirely on free-tier infrastructure.